

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH  
TRƯỜNG ĐẠI HỌC BÁCH KHOA  
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



# KỊCH BẢN DEMO TRỰC TIẾP

## SIMPLE OPERATING SYSTEM SIMULATOR

Phân công, lời trình bày, bảng chứng tính đúng và chuẩn bị phản biện

|                            |                                                                                              |
|----------------------------|----------------------------------------------------------------------------------------------|
| <b>Môn học:</b>            | Hệ điều hành – CO2018                                                                        |
| <b>Lớp:</b>                | L08                                                                                          |
| <b>GVHD:</b>               | Thầy Hoàng Lê Hải Thanh                                                                      |
| <b>Thời lượng dự kiến:</b> | 20–30 phút trình bày và demo trực tiếp                                                       |
| <b>Mục tiêu:</b>           | Chứng minh tính đúng đắn, giải thích thiết kế và thể hiện sự hiểu bài của toàn bộ thành viên |

TP. Hồ Chí Minh, tháng 06/2026

# Mục đích của tài liệu

Tài liệu này là kịch bản vận hành cho buổi báo cáo trực tiếp. Nó không phải một bài đọc thay thế báo cáo, mà là hướng dẫn nhóm phải *làm gì, nói gì, chỉ vào đâu và chứng minh điều gì* trong từng phút. Mọi thành viên phải hiểu phần mình phụ trách và phần tổng thể, bởi giảng viên có thể chuyển câu hỏi từ người đang nói sang bất kỳ thành viên nào.

**Nguyên tắc.** Mục tiêu demo không phải tạo ra nhiều output nhất. Mỗi lệnh được chạy phải gắn với một mệnh đề cần chứng minh, một invariant, một đoạn code chịu trách nhiệm và một giải thích vì sao output là bằng chứng hợp lệ.

## Contents

|                                                 |          |
|-------------------------------------------------|----------|
| Mục đích của tài liệu                           | i        |
| <b>1 Thông điệp trung tâm của buổi demo</b>     | <b>1</b> |
| 1.1 Ba mệnh đề nhóm cần chứng minh . . . . .    | 1        |
| 1.2 Mô hình chứng minh cần sử dụng . . . . .    | 1        |
| <b>2 Phân công tổng thể cho năm thành viên</b>  | <b>1</b> |
| 2.1 Bảng phân công chính . . . . .              | 1        |
| 2.2 Trách nhiệm chung bắt buộc . . . . .        | 2        |
| <b>3 Timeline chuẩn 25 phút</b>                 | <b>2</b> |
| 3.1 Lịch trình chính . . . . .                  | 2        |
| 3.2 Phương án rút gọn 20 phút . . . . .         | 3        |
| 3.3 Phương án đầy đủ 30 phút . . . . .          | 3        |
| <b>4 Chuẩn bị môi trường trước buổi demo</b>    | <b>3</b> |
| 4.1 Bố trí cửa sổ . . . . .                     | 3        |
| 4.2 Lệnh kiểm tra trước khi vào phòng . . . . . | 4        |
| 4.3 Lệnh demo gọn, dễ quan sát . . . . .        | 4        |
| <b>5 Kịch bản của Trần Nhật Bảo Anh</b>         | <b>5</b> |
| 5.1 Mục tiêu phần trình bày . . . . .           | 5        |
| 5.2 Lời mở đầu đề xuất . . . . .                | 5        |
| 5.3 Kiến trúc phải giải thích . . . . .         | 5        |
| 5.4 Demo build và test tổng thể . . . . .       | 5        |
| 5.5 Kiến thức Bảo Anh bắt buộc nắm . . . . .    | 5        |
| 5.6 Câu chuyển giao . . . . .                   | 6        |
| <b>6 Kịch bản của Nguyễn Quốc Trung</b>         | <b>6</b> |
| 6.1 Mệnh đề cần chứng minh . . . . .            | 6        |
| 6.2 Demo 1: priority và quantum . . . . .       | 6        |
| 6.3 Demo 2: weighted slot và reset . . . . .    | 7        |
| 6.4 Demo 3: hai CPU . . . . .                   | 7        |
| 6.5 Source code cần mở . . . . .                | 7        |

|           |                                                        |           |
|-----------|--------------------------------------------------------|-----------|
| 6.6       | Kiến thức Quốc Trung bắt buộc nắm . . . . .            | 7         |
| 6.7       | Câu hỏi dự kiến . . . . .                              | 8         |
| 6.8       | Câu chuyển giao . . . . .                              | 8         |
| <b>7</b>  | <b>Kịch bản của Trần Hoàng Quốc Bảo</b>                | <b>8</b>  |
| 7.1       | Mệnh đề cần chứng minh . . . . .                       | 8         |
| 7.2       | Demo 1: region reuse . . . . .                         | 8         |
| 7.3       | Demo 2: copy theo đúng cú pháp đặc tả . . . . .        | 9         |
| 7.4       | Source code cần mở . . . . .                           | 9         |
| 7.5       | Kiến thức Quốc Bảo bắt buộc nắm . . . . .              | 10        |
| 7.6       | Câu hỏi dự kiến . . . . .                              | 10        |
| 7.7       | Câu chuyển giao . . . . .                              | 10        |
| <b>8</b>  | <b>Kịch bản của Trịnh Quốc An</b>                      | <b>10</b> |
| 8.1       | Mệnh đề cần chứng minh . . . . .                       | 10        |
| 8.2       | Demo 1: syscall table . . . . .                        | 10        |
| 8.3       | Demo 2: cross-PID protection . . . . .                 | 11        |
| 8.4       | Source code cần mở . . . . .                           | 11        |
| 8.5       | Kiến thức Quốc An bắt buộc nắm . . . . .               | 11        |
| 8.6       | Câu hỏi dự kiến . . . . .                              | 12        |
| 8.7       | Câu chuyển giao . . . . .                              | 12        |
| <b>9</b>  | <b>Kịch bản của Nguyễn Trường Giang</b>                | <b>12</b> |
| 9.1       | Mệnh đề cần chứng minh . . . . .                       | 12        |
| 9.2       | Demo MM64 canonical . . . . .                          | 12        |
| 9.3       | Demo semantic assertions . . . . .                     | 13        |
| 9.4       | Source code cần mở . . . . .                           | 13        |
| 9.5       | Kiến thức Trường Giang bắt buộc nắm . . . . .          | 13        |
| 9.6       | Câu hỏi dự kiến . . . . .                              | 14        |
| 9.7       | Câu chuyển giao . . . . .                              | 14        |
| <b>10</b> | <b>Kết luận do Bảo Anh trình bày</b>                   | <b>14</b> |
| 10.1      | Lời kết đề xuất . . . . .                              | 14        |
| <b>11</b> | <b>Ma trận chứng minh tính đúng</b>                    | <b>14</b> |
| <b>12</b> | <b>Kịch bản xử lý sự cố trực tiếp</b>                  | <b>15</b> |
| 12.1      | Nếu output scheduler khác thứ tự đã tập . . . . .      | 15        |
| 12.2      | Nếu một lệnh demo quá dài . . . . .                    | 15        |
| 12.3      | Nếu build lỗi do artifact . . . . .                    | 15        |
| 12.4      | Nếu giảng viên hỏi ngoài phần của thành viên . . . . . | 15        |
| <b>13</b> | <b>Bộ câu hỏi phản biện theo thành viên</b>            | <b>16</b> |
| 13.1      | Câu hỏi dành cho Bảo Anh . . . . .                     | 16        |
| 13.2      | Câu hỏi dành cho Quốc Trung . . . . .                  | 16        |
| 13.3      | Câu hỏi dành cho Quốc Bảo . . . . .                    | 16        |
| 13.4      | Câu hỏi dành cho Quốc An . . . . .                     | 16        |
| 13.5      | Câu hỏi dành cho Trường Giang . . . . .                | 17        |

|                                                     |           |
|-----------------------------------------------------|-----------|
| <b>14 Checklist luyện tập từng thành viên</b>       | <b>17</b> |
| 14.1 Tiêu chí đạt trước buổi demo . . . . .         | 17        |
| 14.2 Kế hoạch tập dượt . . . . .                    | 17        |
| <b>15 Phiếu tự đánh giá sau mỗi lần tập</b>         | <b>18</b> |
| <b>16 Cheat sheet dùng ngay trước khi vào phòng</b> | <b>18</b> |
| <b>17 Kết luận</b>                                  | <b>19</b> |

# 1 Thông điệp trung tâm của buổi demo

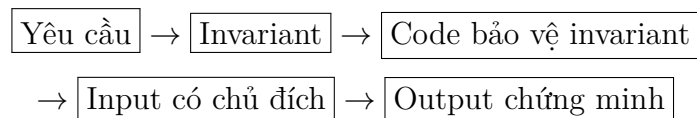
## 1.1 Ba mệnh đề nhóm cần chứng minh

Trong 20–30 phút, nhóm cần làm cho giảng viên tin được ba mệnh đề sau:

1. **Scheduler đúng**: MLQ chọn process theo priority và weighted slot, giữ FIFO trong từng queue, tôn trọng quantum và không giao cùng PCB cho hai CPU.
2. **Memory và syscall đúng**: process thao tác memory qua kernel boundary; allocation/free/read/write/copy bảo toàn dữ liệu và kernel từ chối truy cập frame không thuộc caller.
3. **MM64 đúng**: địa chỉ được tách qua năm cấp, các bảng được cấp sparse, high/low canonical được chấp nhận và non-canonical bị từ chối.

## 1.2 Mô hình chứng minh cần sử dụng

Mỗi demo phải trình bày theo chuỗi:



Ví dụ với cross-PID protection:

- Yêu cầu: process không được truy cập memory của process khác.
- Invariant: physical frame chỉ được raw I/O nếu thuộc page table caller.
- Code: `caller_owns_phyaddr()` trong `src/sys_mem.c`.
- Input: PID 1 cấp frame 0, PID 2 cố đọc và ghi physical address 0.
- Output: hai dòng `Kernel denied PID 2 ....`

**Lưu ý.** Không nói “output giống file mẫu nên code đúng”. Output chỉ là bằng chứng khi nhóm giải thích được input đã cô lập property nào và đoạn code nào đảm bảo property đó.

# 2 Phân công tổng thể cho năm thành viên

## 2.1 Bảng phân công chính

| STT | Thành viên | Vai trò demo | Trách nhiệm chính |
|-----|------------|--------------|-------------------|
|-----|------------|--------------|-------------------|

|   |                                |                                     |                                                                                                                                                                           |
|---|--------------------------------|-------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | Trần Nhật Bảo Anh<br>2410155   | Điều phối và kiến trúc              | Mở đầu; build/test tổng thể; giải thích luồng end-to-end, PCB, syscall boundary; quản lý thời gian; kết luận và nhận câu hỏi tổng hợp.                                    |
| 2 | Nguyễn Quốc Trung<br>2413707   | Scheduler và synchronization        | Demo <code>sched_prio</code> , <code>sched_out_of_slot</code> , <code>sched_2_cpu</code> ; giải thích MLQ, weighted slot, quantum, running list, mutex và timer barrier.  |
| 3 | Trần Hoàng Quốc Bảo<br>2410297 | Virtual memory và tính đúng dữ liệu | Demo <code>os_mem_reuse</code> , <code>os_mem_roundtrip_spec</code> ; giải thích VMA, region, free list, address translation, user/kernel copy và kiểm tra biên.          |
| 4 | Trịnh Quốc An<br>2410046       | System call và protection           | Demo <code>os_syscall_list</code> , <code>os_cross_pid_protection</code> ; giải thích dispatcher, syscall 17, PID lookup, ownership, user/kernel separation và đường lỗi. |
| 5 | Nguyễn Trường Giang<br>2410841 | MM64, kiểm thử và phản biện         | Demo <code>os_mm64_canonical</code> ; giải thích canonical 57-bit, năm cấp, sparse allocation, statistics; trình bày semantic assertions, giới hạn và hướng cải tiến.     |

## 2.2 Trách nhiệm chung bắt buộc

Mỗi thành viên phải có khả năng trả lời ngắn gọn các câu:

1. Simulator gồm những thread và subsystem nào?
2. PCB nằm ở đâu và tại sao phải có running list?
3. Priority nhỏ hơn hay lớn hơn được ưu tiên? Quantum khác weighted slot thế nào?
4. Vì sao memory operation phải đi qua syscall?
5. Virtual address, physical address, page và frame khác nhau thế nào?
6. Test pass chứng minh được gì và chưa chứng minh được gì?

## 3 Timeline chuẩn 25 phút

### 3.1 Lịch trình chính

| Phút | Người phụ trách | Phần | Kết quả cần đạt |
|------|-----------------|------|-----------------|
|------|-----------------|------|-----------------|

|             |                     |                              |                                                                                                                                       |
|-------------|---------------------|------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| 0:00–2:30   | Bảo Anh             | Mở đầu và kiến trúc          | Nêu ba mệnh đề cần chứng minh; vẽ luồng loader–scheduler–CPU–syscall–MM; chạy build/test tổng thể hoặc trình bày kết quả đã chuẩn bị. |
| 2:30–8:00   | Quốc Trung          | Scheduler và synchronization | Chứng minh priority tại biên quantum, weighted slot/reset và không dispatch trùng PCB trên hai CPU.                                   |
| 8:00–13:00  | Quốc Bảo            | Memory correctness           | Chứng minh free-region reuse, copy theo cú pháp đặc tả và roundtrip giữ byte 65; giải thích region/page/frame.                        |
| 13:00–17:00 | Quốc An             | Syscall và protection        | Chứng minh syscall table đúng, kernel lookup caller bằng PID và từ chối raw physical access của PID khác.                             |
| 17:00–21:30 | Trường Giang        | MM64 và kiểm thử             | Chứng minh năm cấp, sparse allocation, canonical validation và ý nghĩa statistics/semantic assertions.                                |
| 21:30–24:00 | Bảo Anh + cả nhóm   | Kết luận và giới hạn         | Tổng hợp traceability, nói thẳng giới hạn, hướng cải tiến và mời phản biện.                                                           |
| 24:00–30:00 | Thành viên được hỏi | Q&A                          | Trả lời theo bốn lớp: định nghĩa, code, bằng chứng, trade-off.                                                                        |

## 3.2 Phương án rút gọn 20 phút

Nếu thời gian trình bày chỉ có 20 phút:

- Không chạy `sched_out_of_slot`; dùng bảng quota 3 : 2 : 1 để giải thích miệng.
- Gộp `os_syscall_list` vào phần giới thiệu, chỉ chiếu output đã chuẩn bị.
- Mỗi người nói tối đa ba phút; phần Q&A dành ít nhất bốn phút.
- Vẫn phải chạy trực tiếp bốn demo cốt lõi: `sched_2_cpu`, `os_mem_roundtrip_spec`, `os_cross_pid_protection`, `os_mm64_canonical`.

## 3.3 Phương án đầy đủ 30 phút

Nếu có đủ 30 phút, chạy cả sáu demo chính, mở source code tại các hàm trọng tâm, và dành 6–8 phút cuối cho câu hỏi phản biện. Không nên thêm testcase khác trừ khi giảng viên yêu cầu.

# 4 Chuẩn bị môi trường trước buổi demo

## 4.1 Bố trí cửa sổ

Nên chuẩn bị ba cửa sổ:

1. Terminal chính, font lớn, dùng để chạy lệnh.
2. IDE mở sẵn `src/sched.c`, `src/sys_mem.c`, `src/libmem.c` và `src/mm64.c`. Mở thêm `src/timer.c` hoặc `src/loader.c` khi được hỏi sâu.
3. PDF báo cáo hoặc study guide tại các sơ đồ và bảng bằng chứng.

Tắt notification, đóng tab cá nhân, tăng kích thước terminal và đảm bảo đường dẫn repository không bị đổi.

## 4.2 Lệnh kiểm tra trước khi vào phòng

```
1 pwd
2 git status --short
3 make clean
4 make all
5 make test
```

Kết quả bắt buộc trước khi demo:

```
1 All semantic output assertions passed.
```

Sau đó chạy thử từng lệnh demo:

```
1 ./os sched_prio
2 ./os sched_out_of_slot
3 ./os sched_2_cpu
4 ./os os_mem_reuse
5 ./os os_mem_roundtrip_spec
6 ./os os_syscall_list
7 ./os os_cross_pid_protection
8 ./os os_mm64_canonical
```

## 4.3 Lệnh demo gọn, dễ quan sát

Để output không tràn màn hình, có thể lọc đúng bằng chứng:

```
1 ./os sched_prio | grep -E "Loaded|Dispatched|Put process|finished"
2
3 ./os sched_2_cpu | grep -E "CPU [01]: (Dispatched|Put|Processed)"
4
5 ./os os_mem_reuse | grep -E "MEM alloc|MEM free|MEM read"
6
7 ./os os_mem_roundtrip_spec | grep -E "MEM alloc|MEM read"
8
9 ./os os_cross_pid_protection | grep -E "MEM alloc|Kernel denied"
10
11 ./os os_mm64_canonical | grep -E "MM64 mapped|MM64 pid|MM64 rejected"
```

**Lưu ý.** Luôn giải thích rằng `grep` chỉ lọc output để dễ nhìn, không thay đổi chương trình hoặc tiêu chí đúng. Khi giảng viên yêu cầu, chạy lại không lọc.



## 5 Kịch bản của Trần Nhật Bảo Anh

### 5.1 Mục tiêu phần trình bày

Trong 2–3 phút đầu, Bảo Anh phải tạo được “bản đồ” để mọi demo sau có chỗ gắn vào. Không đi sâu code ngay; tập trung vào vấn đề, kiến trúc và cách nhóm chứng minh.

### 5.2 Lời mở đầu đề xuất

**Lời trình bày:** “Nhóm em xây dựng một Simple Operating System Simulator gồm ba trục chính: scheduler và synchronization để chia sẻ CPU, virtual memory để quản lý và cô lập RAM, cùng system-call interface để user process không thao tác trực tiếp tài nguyên kernel. Trong phần demo, nhóm em không chỉ chạy output mà sẽ chứng minh từng invariant bằng input có chủ đích, đoạn code bảo vệ invariant và semantic assertion tương ứng.”

**Lời trình bày:** “Ba mệnh đề nhóm em sẽ chứng minh là: scheduler chọn đúng và không giao trùng PCB; memory operation bảo toàn dữ liệu và bảo vệ ownership; cuối cùng MM64 page-table walk chấp nhận đúng canonical address và cấp bảng sparse.”

### 5.3 Kiến trúc phải giải thích

Bảo Anh cần trình bày luồng:

**Configuration** → Loader tạo PCB → MLQ ready queues  
→ CPU chạy instruction → System call → Virtual/physical memory

Timer thread tạo barrier theo time slot giữa loader và các CPU. Khi process hết quantum, PCB quay về ready queue; khi hoàn tất, memory context và PCB được giải phóng.

### 5.4 Demo build và test tổng thể

```
1 make clean && make all && make test
```

**Lời trình bày:** “Lệnh này chứng minh repository có thể build từ trạng thái sạch và toàn bộ 23 configuration chạy đến cuối. Tuy nhiên, exit code bằng 0 chưa đủ chứng minh tính đúng. Cuối suite, `tests/assert_outputs.sh` kiểm tra các semantic property như cross-PID denial, region reuse, canonical validation và byte roundtrip. Các thành viên tiếp theo sẽ giải thích từng property.”

**Điểm chứng minh:** Build sạch chứng minh tính tái lập; semantic assertions chứng minh một tập invariant đã xác định. Nó không chứng minh chương trình không còn mọi bug, và nhóm phải nói rõ giới hạn này.

### 5.5 Kiến thức Bảo Anh bắt buộc nắm

- Vai trò `main()`, loader thread, CPU thread, timer và kernel structure.

- Vòng đời PCB: NEW, READY, RUNNING, FINISHED.
- Luồng một ALLOC từ CPU instruction đến syscall 17 và memory allocator.
- Khác biệt giữa simulation và OS thật.
- Traceability từ yêu cầu đến testcase và assertion.
- Giới hạn chính: queue mảng  $O(n)$ , PID lookup tuyến tính, global memory lock, FIFO replacement, không có TLB/hardware interrupt thật.

## 5.6 Câu chuyển giao

**Lời trình bày:** “Sau khi đã xác định luồng tổng thể, bạn Quốc Trung sẽ chứng minh scheduler giữ đúng priority, weighted quota và ownership PCB khi chạy đa CPU.”

# 6 Kịch bản của Nguyễn Quốc Trung

## 6.1 Mệnh đề cần chứng minh

1. Priority nhỏ hơn được ưu tiên tại scheduling boundary.
2. Process mới không preempt giữa quantum hiện tại.
3. Weighted slot cho queue quota khác nhau và reset khi cần.
4. Hai CPU không nhận cùng PCB.

## 6.2 Demo 1: priority và quantum

Mở input/sched\_prio:

```
1 2 1 2
2 0 s1 1
3 1 s0 0
```

**Lời trình bày:** “Quantum là 2, có một CPU. PID 1 priority 1 đến lúc 0; PID 2 priority 0 đến lúc 1. PID 2 có priority cao hơn nhưng không cướp CPU giữa quantum. Sau khi PID 1 hết quantum và được put lại queue, scheduler mới chọn PID 2.”

Chạy:

```
1 ./os sched_prio | grep -E "Loaded|Dispatched|Put process|finished"
```

Chỉ vào thứ tự:

```
1 CPU 0: Dispatched process 1
2 Loaded ... PID: 2 PRI0: 0
3 CPU 0: Put process 1 to run queue
4 CPU 0: Dispatched process 2
```

**Điểm chứng minh:** Input cố ý cho process priority cao đến trong lúc PID 1 đang chạy. Output chứng minh policy là preemption tại biên quantum, không phải preemption tức thời.

### 6.3 Demo 2: weighted slot và reset

Mở input/sched\_out\_of\_slot. Giải thích:

$$\text{slot}[137] = 3, \quad \text{slot}[138] = 2, \quad \text{slot}[139] = 1$$

Chạy:

```
1 ./os sched_out_of_slot | grep -E "Dispatched|Put process|finished"
```

**Lời trình bày:** “Sau lần chạy đầu của priority 139, queue này hết quota. Priority 137 được chọn ba quantum, priority 138 hai quantum. Khi các queue có PCB đều hết quota, scheduler reset chu kỳ. Process priority thấp không bị mất; nó tiếp tục sau khi các process cao hơn hoàn tất hoặc quota cho phép.”

### 6.4 Demo 3: hai CPU

Chạy:

```
1 ./os sched_2_cpu | grep -E "CPU [01]: (Dispatched|Put|Processed)"
```

**Lời trình bày:** “Điều cần chứng minh không phải CPU 0 bắt buộc nhận PID cụ thể nào, vì thứ tự pthread có thể thay đổi. Invariant là tại cùng thời điểm, CPU 0 và CPU 1 không dispatch cùng PID. `get_proc()` dequeue PCB và thêm vào `running_list` trong cùng `queue_lock`, nên CPU thứ hai không còn nhìn thấy PCB đó trong ready queue.”

### 6.5 Source code cần mở

Trong `src/sched.c`, chỉ vào:

- `mlq_ready_queue[MAX_PRIO]` và `slot[MAX_PRIO]`.
- `get_mlq_proc()`: quét priority, giảm quota, reset khi không chọn được.
- `put_mlq_proc()`: purge running list rồi enqueue lại.
- `queue_lock`: bao trọn thao tác chuyển ownership.

Trong `src/timer.c`, chỉ vào `next_slot()` và giải thích condition variables tạo barrier.

### 6.6 Kiến thức Quốc Trung bắt buộc nắm

- Priority, quantum, weighted slot và FIFO khác nhau thế nào.
- Tại sao queue-level FIFO nằm ở `dequeue()`, còn priority selection nằm ở scheduler.
- Running list giải quyết vấn đề gì.
- Race condition nếu bỏ `queue_lock`.
- Mutex khác condition variable thế nào.

- Starvation có được loại bỏ hoàn toàn hay không.
- Độ phức tạp quét MLQ, enqueue, dequeue và PID lookup.

## 6.7 Câu hỏi dự kiến

**Hỏi:** Vì sao process priority thấp vẫn phải chờ rất lâu?

**Đáp:** Weighted slot giảm starvation nhưng quota của priority cao lớn. Nếu queue cao liên tục có việc, thời gian chờ queue thấp vẫn dài. Đây là trade-off giữa response time theo priority và fairness.

**Hỏi:** Nếu bỏ running list thì sao?

**Đáp:** Kernel không có registry cho process đang chạy để lookup PID; ownership giữa ready/running cũng khó kiểm chứng và có nguy cơ dispatch trùng.

## 6.8 Câu chuyển giao

**Lời trình bày:** “Scheduler đã đảm bảo mỗi thời điểm process được điều phối đúng. Tiếp theo, bạn Quốc Bảo sẽ chứng minh address space của process quản lý region đúng và dữ liệu được bảo toàn qua ranh giới user/kernel.”

# 7 Kịch bản của Trần Hoàng Quốc Bảo

## 7.1 Mệnh đề cần chứng minh

1. Allocation tạo region hợp lệ và free region được tái sử dụng.
2. Access kiểm tra đúng biên half-open  $[start, end)$ .
3. Copy user–kernel–user theo cú pháp đặc tả bảo toàn byte dữ liệu.
4. Region logic, page và frame vật lý là các lớp khác nhau.

## 7.2 Demo 1: region reuse

Mở input/proc/memreuse:

```
1 20 4
2 alloc 100 1
3 free 1
4 alloc 50 2
5 read 2 49 3
```

Chạy:

```
1 ./os os_mem_reuse | grep -E "MEM alloc|MEM free|MEM read"
```

Output trọng tâm:

```
1 MEM alloc PID 1 region 1 [0,100) size 100
2 MEM free PID 1 region 1 [0,100)
3 MEM alloc PID 1 region 2 [0,50) size 50
4 MEM read PID 1 region 2 offset 49 value 0
```

**Lời trình bày:** “Region 1 được free thành khoảng trống [0,100). Allocation 50 byte sau đó nhận lại [0,50), chứng minh allocator tìm free region trước khi tăng `sbrk`. Offset 49 là byte cuối hợp lệ của region 50 byte; offset 50 phải bị từ chối theo interval half-open.”

**Điểm chứng minh:** Đây là bằng chứng tái sử dụng logical region. Không nên nói rằng frame vật lý chắc chắn đã unmap rồi cấp lại, vì implementation có thể giữ page mapping trong address space process.

## 7.3 Demo 2: copy theo đúng cú pháp đặc tả

Mở `input/proc/memroundtrip_spec`:

```
1 20 7
2 alloc 16 1
3 write 65 1 0
4 copy_from_user 1 2
5 alloc 16 3
6 copy_to_user 2 3 0
7 read 3 0 4
8 calc
```

Chạy:

```
1 ./os os_mem_roundtrip_spec | grep -E "MEM alloc|MEM read"
```

**Lời trình bày:** “Lệnh `copy_from_user 1 2` và `copy_to_user 2 3 0` dùng đúng số tham số trong đặc tả. Parser mặc định copy một byte. Byte 65 được ghi ở user region 1, copy sang kernel region 2, copy về user region 3 và đọc lại vẫn là 65. Đây là bằng chứng end-to-end cho parser, syscall, helper user/kernel và address translation.”

## 7.4 Source code cần mở

Trong `src/loader.c`, chỉ vào parser tương thích:

- `copy_from_user` hai tham số mặc định offset 0, size 1.
- `copy_to_user` ba tham số mặc định size 1.
- Dạng bốn tham số vẫn được giữ để copy nhiều byte.

Trong `src/libmem.c`, chỉ vào:

- `__alloc()`: tìm free region hoặc tăng VMA.
- `__free()`: đưa region vào free list.
- `__kmem_copy_from_user()` và `__kmem_copy_to_user()`: copy từng byte qua helper có kiểm tra.

## 7.5 Kiến thức Quốc Bảo bắt buộc nắm

- Virtual address, page number, offset, frame number và physical address.
- VMA, `sbrk`, symbol region table và free-region list.
- Half-open interval và kiểm tra biên.
- Internal/external fragmentation.
- Paging so với continuous allocation.
- User memory, kernel memory, `kmalloc` và cache pool.
- Tại sao copy helper an toàn hơn dereference user pointer trực tiếp.

## 7.6 Câu hỏi dự kiến

**Hỏi:** ALLOC 100 byte có dùng đúng 100 byte RAM không?

**Đáp:** Region logic dài 100 byte, nhưng mapping vật lý align theo page size nên có thể dùng một page lớn hơn. Phần dư có thể được đưa vào free list.

**Hỏi:** FREE có luôn trả frame vật lý ngay không?

**Đáp:** Không trong user-region path hiện tại. Nó trả logical region cho free list để tái sử dụng; page mapping có thể còn tồn tại đến khi process kết thúc.

## 7.7 Câu chuyển giao

**Lời trình bày:** “Phần vừa rồi chứng minh dữ liệu và region được quản lý đúng trong address space của process. Bạn Quốc An sẽ tiếp tục chứng minh process không thể vượt qua kernel boundary để truy cập frame của process khác.”

# 8 Kịch bản của Trịnh Quốc An

## 8.1 Mệnh đề cần chứng minh

1. Syscall table ánh xạ đúng số syscall đến handler.
2. Kernel xác định caller bằng PID từ registry do kernel quản lý.
3. Raw physical I/O bị từ chối nếu frame không thuộc caller.
4. Unified syscall tập trung policy nhưng có trade-off.

## 8.2 Demo 1: syscall table

Chạy:

```
1 ./os os_syscall_list | grep -E "0-sys|17-sys|440-sys"
```

Output:

```
1 0-sys_listsyscall
2 17-sys_memmap
3 440-sys_xxxhandler
```

**Lời trình bày:** “src/syscall.tbl là nguồn dữ liệu duy nhất. Script sinh `syscalltbl.lst`; `src/syscall.c` include danh sách này bằng X-macro để tạo prototype, bảng tên và dispatcher switch. Điều này tránh ba nơi khai báo bị lệch nhau.”

### 8.3 Demo 2: cross-PID protection

Mở input owner/intruder và chạy:

```
1 ./os os_cross_pid_protection | grep -E "MEM alloc|Kernel denied"
```

Output:

```
1 MEM alloc PID 1 region 1 [0,100) size 100
2 Kernel denied PID 2 physical read at 0x0
3 Kernel denied PID 2 physical write at 0x0
```

**Lời trình bày:** “PID 1 sở hữu frame vật lý chứa region đầu tiên. PID 2 biết địa chỉ 0 và gọi trực tiếp physical read/write, nhưng biết địa chỉ không tạo ra quyền. Handler syscall lookup PCB thật của PID 2 rồi kiểm tra frame có xuất hiện trong page table PID 2 hay không. Vì không thuộc caller, cả hai thao tác bị từ chối.”

### 8.4 Source code cần mở

Trong `src/syscall.c`:

- `_syscall()` dispatch theo số syscall.
- Unknown syscall đi vào `__sys_ni_syscall()`.

Trong `src/sys_mem.c`:

- `find_proc_by_pid(pid)` xác định caller.
- `caller_owns_phyaddr()` kiểm tra ownership.
- Các operation selector của syscall 17.

Trong `src/sched.c`, chỉ vào `find_proc_by_pid()` duyệt running list và ready queues dưới `queue_lock`.

### 8.5 Kiến thức Quốc An bắt buộc nắm

- User mode/kernel mode và lý do cần syscall.
- Dispatcher, syscall number, register arguments và handler.
- Vì sao không truyền trực tiếp `pcb_t *` từ user.

- Address translation khác access control thế nào.
- Frame ownership và cross-PID isolation.
- Ưu/nhược điểm unified syscall.
- Hậu quả nếu syscall dài hoặc thiếu validation.

## 8.6 Câu hỏi dự kiến

**Hỏi:** Vì sao PID cũng có thể bị giả mạo?

**Đáp:** Trong OS thật, kernel lấy identity từ execution context/trap, không tin PID do user tự khai. Simulator truyền PID từ PCB đang chạy qua library; handler vẫn lookup PCB trong registry kernel thay vì nhận con trỏ PCB tùy ý. Đây là mô phỏng đơn giản hóa nguyên tắc đó.

**Hỏi:** Dịch được physical address có nghĩa được phép truy cập không?

**Đáp:** Không. Translation trả lời dữ liệu nằm đâu; access control trả lời caller có quyền hay không. Test cross-PID cô lập chính hai khái niệm này.

## 8.7 Câu chuyển giao

**Lời trình bày:** “Kernel boundary đã bảo vệ ownership. Phần cuối, bạn Trường Giang sẽ chứng minh page-table hierarchy 64-bit xử lý đúng canonical address và cấp bảng sparse, đồng thời giải thích cách nhóm kiểm thử các invariant.”

# 9 Kịch bản của Nguyễn Trường Giang

## 9.1 Mệnh đề cần chứng minh

1. Địa chỉ 57-bit được tách thành năm index 9 bit và offset 12 bit.
2. Low và high canonical address được chấp nhận.
3. Non-canonical address bị từ chối trước page-table walk.
4. Page-table branch được cấp sparse và statistics phản ánh đúng chi phí.

## 9.2 Demo MM64 canonical

Chạy:

```
1 ./os os_mm64_canonical | grep -E "MM64 mapped|MM64 pid|MM64 rejected"
```

Giải thích từng mốc:

| Địa chỉ | Indexes | Ý nghĩa |
|---------|---------|---------|
|---------|---------|---------|



|                    |                         |                                                                            |
|--------------------|-------------------------|----------------------------------------------------------------------------|
| 0x0                | [0, 0, 0, 0, 0]         | Low canonical; walk đầu cấp bốn bảng con, tổng bytes thành 20480.          |
| 0x200000           | [0, 0, 0, 1, 0]         | Khác PMD index, cần thêm một PT table; bytes tăng lên 24576.               |
| 0xff00000000000000 | [256, 0, 0, 0, 0]       | High canonical, tạo PGD branch mới; bytes tăng lên 40960.                  |
| 0x1000000000000000 | Không có index/walk mới | Bit 56 bằng 1 nhưng các bit trên không sign-extend; bị từ chối trước walk. |

**Lời trình bày:** “Mỗi bảng có 512 entry, mỗi entry 8 byte nên đúng 4096 byte. Root PGD đã có từ lúc init; mapping đầu cần thêm P4D, PUD, PMD và PT, do đó  $5 \times 4096 = 20480$ . Các địa chỉ chia sẻ prefix dùng lại bảng cũ, thể hiện sparse allocation.”

### 9.3 Demo semantic assertions

Mở `tests/assert_outputs.sh` và trình bày:

**Lời trình bày:** “Nhóm phân biệt ba tầng: build, execution và semantic assertion. Chạy không crash chỉ chứng minh execution. Script này kiểm tra các mốc có ý nghĩa như frame của PID khác bị từ chối, region được reuse, high canonical được chấp nhận, non-canonical bị từ chối và byte roundtrip bằng 65.”

### 9.4 Source code cần mở

Trong `src/mm64.c`, chỉ vào:

- `paging64_is_canonical()`.
- `walk_pte()` và tham số `create`.
- `alloc_table()` tăng `pgtable_bytes`.
- `get_pd_from_address()` tách index.

Trong `include/mm64.h`, chỉ vào bit range PGD/P4D/PUD/PMD/PT.

### 9.5 Kiến thức Trường Giang bắt buộc nắm

- Vì sao 512 entry tương ứng 9 bit.
- Vì sao page-table page là 4096 byte.
- Canonical sign extension của bit 56.
- Sparse allocation và lợi ích so với page table phẳng.
- Page-table walk cost và vai trò TLB trong OS thật.

- Ý nghĩa walks, accesses, bytes.
- Test hữu hạn chứng minh gì và không chứng minh gì.
- Giới hạn test hiện tại và đề xuất cải tiến.

## 9.6 Câu hỏi dự kiến

**Hỏi:** Page table phẳng 57-bit tốn bao nhiêu?

**Đáp:** Page 4 KiB dùng 12-bit offset, còn 45-bit page number, tức  $2^{45}$  PTE. Với 8 byte/PTE, bảng phẳng cần  $2^{48}$  byte, tương đương 256 TiB cho một address space, nên hierarchical sparse paging là cần thiết.

**Hỏi:** Tại sao test pass chưa đủ?

**Đáp:** Test chỉ chứng minh property được assert trên input đã chạy. Các đường stress, race hiếm, swap-full và invalid argument vẫn cần thêm kiểm thử hoặc công cụ như sanitizer/race detector.

## 9.7 Câu chuyển giao

**Lời trình bày:** “Các demo vừa rồi nối từ policy đến code và output: scheduler giữ ownership CPU, syscall giữ ranh giới đặc quyền, memory giữ dữ liệu và MM64 giữ đúng translation. Bạn Bảo Anh sẽ tổng hợp các giới hạn và kết luận.”

# 10 Kết luận do Bảo Anh trình bày

## 10.1 Lời kết đề xuất

**Lời trình bày:** “Qua sáu kịch bản có chủ đích, nhóm em đã chứng minh ba nhóm invariant: scheduler tôn trọng priority/quantum và không dispatch trùng PCB; memory operation bảo toàn region, dữ liệu và frame ownership qua syscall boundary; MM64 chấp nhận đúng canonical address, từ chối địa chỉ sai và cấp page table sparse. Nhóm cũng dùng semantic assertions để biến các bằng chứng này thành kiểm thử có thể lặp lại.”

**Lời trình bày:** “Nhóm em không xem test pass là bằng chứng tuyệt đối. Các giới hạn còn lại gồm global lock làm giảm concurrency, FIFO replacement đơn giản, PID lookup tuyến tính, chưa mô phỏng TLB và chưa stress mọi đường swap/out-of-memory. Nếu mở rộng, nhóm sẽ thêm lock theo address space, PID registry hiệu quả hơn, stress/race tests và replacement policy gần OS thật hơn.”

# 11 Ma trận chứng minh tính đúng

| Property                      | Input/demo | Code chịu trách nhiệm            | Bảng chứng quan sát                                   |
|-------------------------------|------------|----------------------------------|-------------------------------------------------------|
| Priority tại quantum boundary | sched_prio | get_mfq_proc(),<br>cpu_routine() | PID 2 priority 0 chỉ dispatch sau khi PID 1 được put. |

|                          |                         |                                           |                                                                    |
|--------------------------|-------------------------|-------------------------------------------|--------------------------------------------------------------------|
| Weighted quota/reset     | sched_out_of_slot       | slot[], reset loop trong get_mfq_proc()   | Priority 137 chạy ba quota, 138 hai quota, process 139 không mất.  |
| Không dispatch trùng PCB | sched_2_cpu             | queue_lock, running_list                  | Hai CPU luôn nhận PID khác nhau.                                   |
| Free-region reuse        | os_mem_reuse            | __free(), get_free_vmrq_area()            | Region 2 nhận lại [0, 50) từ vùng [0, 100) vừa free.               |
| Spec-compatible copy     | os_mem_roundtrip_spec   | Parser loader, copy helpers, syscall 17   | Cú pháp đặc tả chạy được; byte đọc lại bằng 65.                    |
| Cross-PID isolation      | os_cross_pid_protection | find_proc_by_pid(), caller_owns_phyaddr() | PID 2 bị từ chối read/write frame 0 thuộc PID 1.                   |
| Syscall mapping          | os_syscall_list         | syscall.tbl, generated list, dispatcher   | Hiện đúng syscall 0, 17 và 440.                                    |
| Canonical MM64           | os_mm64_canonical       | paging64_is_canonical(), walk_pte()       | Low/high accepted; non-canonical rejected; bytes tăng theo branch. |

## 12 Kịch bản xử lý sự cố trực tiếp

### 12.1 Nếu output scheduler khác thứ tự đã tập

Không hoảng hoặc khẳng định chương trình sai. Nói:

**Lời trình bày:** “Một số dòng độc lập có thể đổi thứ tự do pthread scheduling. Nhóm em sẽ kiểm tra invariant thay vì so toàn bộ trace: tại cùng thời điểm không có hai CPU giữ cùng PID, và priority được tôn trọng tại scheduling boundary.”

Sau đó chạy lại với **grep** hoặc dùng output đã lưu để giải thích.

### 12.2 Nếu một lệnh demo quá dài

Dừng bằng **Ctrl-C** chỉ khi thực sự cần, sau đó dùng output đã sinh bởi **make test**:

```
1 grep -E "PATTERN" output/test_name.output
```

Giải thích rõ đây là output từ suite vừa chạy, không phải output dựng sẵn.

### 12.3 Nếu build lỗi do artifact

```
1 make clean
2 make all
```

Không sửa code trực tiếp trong buổi demo trừ khi giảng viên yêu cầu. Một thay đổi vội có thể phá toàn bộ môi trường.

### 12.4 Nếu giảng viên hỏi ngoài phần của thành viên

Người được hỏi trả lời phần mình biết trước. Nếu cần chuyển:

**Lời trình bày:** “Phần invariant tổng thể em hiểu là ...; chi tiết implementation nằm trong module do bạn ... phụ trách, em xin phép để bạn ấy bổ sung chính xác đoạn code.”

Không im lặng hoàn toàn và không đẩy ngay câu hỏi cho người khác.

## 13 Bộ câu hỏi phản biện theo thành viên

### 13.1 Câu hỏi dành cho Bảo Anh

1. Tại sao simulator cần cả loader, timer và CPU thread?
2. Luồng một instruction ALLOC đi qua những hàm nào?
3. Bằng chứng nào chứng minh implementation bám đặc tả?
4. Những phần nào là mở rộng so với source gốc?
5. Vì sao test pass không phải bằng chứng tuyệt đối?

### 13.2 Câu hỏi dành cho Quốc Trung

1. Quantum khác weighted slot thế nào?
2. Vì sao priority cao không preempt tức thời?
3. Nếu bỏ queue lock, lỗi nào có thể xảy ra?
4. Running list có vai trò gì ngoài logging?
5. Scheduler có đảm bảo fairness tuyệt đối không?

### 13.3 Câu hỏi dành cho Quốc Bảo

1. VMA, region, page và frame khác nhau thế nào?
2. Tại sao offset bằng size không hợp lệ?
3. FREE có trả physical frame ngay không?
4. Copy user/kernel kiểm tra biên ở đâu?
5. Continuous allocation có ưu/nhược điểm gì so với paging?

### 13.4 Câu hỏi dành cho Quốc An

1. Vì sao user không được truyền PCB pointer qua syscall?
2. PID lookup tìm process đang chạy ở đâu?
3. Translation khác authorization thế nào?
4. Unified syscall có nhược điểm gì?
5. Điều gì xảy ra nếu unknown syscall number được gọi?

## 13.5 Câu hỏi dành cho Trường Giang

1. Vì sao canonical address cần sign extension?
2. Vì sao mỗi cấp dùng 9 bit?
3. Vì sao first mapping dùng 20480 byte?
4. Sparse allocation tiết kiệm khi nào?
5. Statistics hiện tại chưa đo được điều gì?

## 14 Checklist luyện tập từng thành viên

### 14.1 Tiêu chí đạt trước buổi demo

Mỗi thành viên chỉ được xem là sẵn sàng khi:

- Có thể trình bày phần mình trong thời gian quy định mà không đọc nguyên văn.
- Có thể chạy đúng lệnh demo và chỉ ra ba dòng output quan trọng.
- Có thể mở đúng file/hàm trong vòng 15 giây.
- Có thể giải thích một invariant và một trade-off.
- Trả lời được toàn bộ câu hỏi chung và ít nhất bốn câu hỏi chuyên môn.
- Biết xử lý khi output concurrent đổi thứ tự.

### 14.2 Kế hoạch tập dượt

| Lần | Mục tiêu       | Cách thực hiện                                                                                |
|-----|----------------|-----------------------------------------------------------------------------------------------|
| 1   | Hiểu nội dung  | Mỗi người trình bày không giới hạn thời gian; thành viên khác hỏi “vì sao” sau từng kết luận. |
| 2   | Đúng kỹ thuật  | Chạy toàn bộ lệnh demo từ môi trường sạch; ghi lại lỗi thao tác và điểm chuyển cửa sổ chậm.   |
| 3   | Đúng thời gian | Bấm giờ 25 phút; cắt mọi câu không phục vụ ba mệnh đề trung tâm.                              |
| 4   | Phản biện chéo | Mỗi người hỏi phần của người khác; người bị hỏi phải trả lời trước khi chủ phần bổ sung.      |
| 5   | Tổng duyệt     | Mô phỏng phòng báo cáo, dùng máy sẽ mang đi, không dùng giữa chừng trừ lỗi thực sự.           |

## 15 Phiếu tự đánh giá sau mỗi lần tập

| Tiêu chí                                                   | Tốt | Đạt | Chưa đạt | Ghi chú |
|------------------------------------------------------------|-----|-----|----------|---------|
| Nói rõ yêu cầu/invariant trước khi chạy lệnh               |     |     |          |         |
| Giải thích được input được thiết kế để cô lập property nào |     |     |          |         |
| Chỉ đúng code chịu trách nhiệm                             |     |     |          |         |
| Chỉ đúng output chứng minh                                 |     |     |          |         |
| Không nhầm priority, quantum và weighted slot              |     |     |          |         |
| Không nhầm region, page và frame                           |     |     |          |         |
| Không khẳng định quá mức những gì test chứng minh          |     |     |          |         |
| Trả lời được trade-off và giới hạn                         |     |     |          |         |
| Chuyển giao giữa thành viên tự nhiên                       |     |     |          |         |
| Hoàn thành trong thời gian                                 |     |     |          |         |

## 16 Cheat sheet dùng ngay trước khi vào phòng

| Chủ đề          | Một câu phải nhớ                                                                                        |
|-----------------|---------------------------------------------------------------------------------------------------------|
| Thông điệp      | Mỗi output phải nối với invariant và code, không chỉ nói “test pass”.                                   |
| Scheduler       | Priority nhỏ hơn cao hơn; weighted slot là $140 - prio$ ; quantum giới hạn thời gian giữ CPU.           |
| Đa CPU          | <code>queue_lock</code> bảo vệ chuyển PCB từ ready queue sang running list.                             |
| Synchronization | Mutex bảo vệ critical section; condition variable giúp thread ngủ chờ condition/time slot.              |
| Memory          | Region là allocation logic; page map sang frame; interval là $[start, end)$ .                           |
| Syscall         | Kernel lookup caller bằng PID và kiểm tra ownership trước raw physical I/O.                             |
| Copy spec       | <code>copy_from_user src dst</code> và <code>copy_to_user src dst offset</code> mặc định copy một byte. |
| MM64            | 57 bit bằng năm index 9 bit cộng offset 12 bit; bit 56 phải được sign-extend.                           |

|            |                                                                                            |
|------------|--------------------------------------------------------------------------------------------|
| Statistics | Một bảng là 4096 byte; first branch gồm root và bốn bảng con = 20480 byte.                 |
| Giới hạn   | Test hữu hạn không chứng minh không còn bug; nói rõ stress/race và swap-full chưa phủ hết. |

## 17 Kết luận

Một buổi demo tốt không phải là buổi có nhiều lệnh nhất, mà là buổi mỗi thành viên đều cho thấy mình hiểu quan hệ giữa yêu cầu, thiết kế, code và bằng chứng. Nhóm cần giữ nhịp trình bày ngắn, trực tiếp và trung thực về giới hạn. Khi bị hỏi sâu, hãy trả lời theo bốn lớp:

Định nghĩa → Implementation → Bằng chứng → Trade-off

Nếu cả năm thành viên đều làm được điều này, nhóm không chỉ chứng minh chương trình chạy đúng trên các testcase đã chọn, mà còn chứng minh được sự hiểu bài và khả năng bảo vệ các quyết định kỹ thuật của mình.